Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets (ServiceNow)

1. ABSTRACT

ServiceNow is a cloud-based platform widely used for IT service management and workflow automation. Ensuring data accuracy and managing configuration changes across multiple environments are essential aspects of maintaining a reliable ServiceNow system. This project focuses on enforcing mandatory fields using UI Policies and migrating configuration changes using Update Sets.

UI Policies allow administrators to dynamically control form behavior by making fields mandatory, read-only, or visible based on specific conditions. This improves data integrity and ensures users provide necessary information while submitting forms. Update Sets are used to track and migrate configuration changes from one ServiceNow instance to another, such as from development to testing or production environments.

The project demonstrates how these features improve system reliability, streamline development workflows, and ensure consistent configuration across instances.

2. INTRODUCTION

ServiceNow provides a powerful platform for automating IT service management processes and improving organizational productivity. One common challenge in enterprise systems is maintaining data quality and ensuring that required information is always captured in forms.

UI Policies are designed to control the behavior of form fields dynamically without requiring scripting. They help enforce validation rules and improve the user experience by guiding users to enter correct information.

Another important aspect of ServiceNow development is managing configuration changes across different environments. Update Sets provide a mechanism for capturing and migrating these changes safely.

This project focuses on implementing UI Policies to enforce mandatory fields and using Update Sets to migrate these configurations between ServiceNow instances.

3. SYSTEM REQUIREMENTS

Hardware Requirements

Computer/Laptop

Minimum 4 GB RAM

Stable Internet Connection

Software Requirements

ServiceNow Developer Instance

Web Browser (Chrome, Firefox, Edge)

ServiceNow Platform Access

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Many organizations face problems such as incomplete data entry in forms and difficulty managing configuration changes across development environments. These issues can lead to inaccurate data, inefficient workflows, and deployment errors.

The proposed solution addresses these challenges using UI Policies and Update Sets within the ServiceNow platform.

Problems

Users submit forms without filling required fields

Manual validation of form data

Difficulty migrating configuration changes

Inconsistent system configuration across instances

Solutions

Enforce mandatory fields using UI Policies

Dynamic form validation based on conditions

Track configuration changes using Update Sets

Migrate changes safely between ServiceNow instances

4.2 Proposed Solution

Functional Requirements

Parameter

Description

Problem

Users may submit incomplete form data

Solution

Enforce mandatory fields using UI Policies

Migration Issue

Configuration changes must be moved between instances

Solution Approach

Use Update Sets to migrate changes

Automation

Automatic field validation and change tracking

System Impact

Improved data accuracy and deployment management

Scalability

Applicable to multiple tables and forms

4.3 Solution Architecture

The system architecture includes ServiceNow forms, UI Policies, and Update Set mechanisms that ensure proper validation and configuration management.

Components

1. User Interface Users interact with forms such as Incident, Change Request, or Request Items.

2. UI Policy Engine Applies conditions and automatically enforces field behavior such as mandatory or read-only.

3. Configuration Layer Captures system configurations like UI Policies, forms, and scripts.

4. Update Set Manager Tracks and groups configuration changes into update sets.

5. Instance Migration Workflow

Development Instance

↓

Export Update Set (XML)

↓

Test Instance (Preview & Commit)

↓

Production Instance

5. IMPLEMENTATION

Creating UI Policy

Steps:

Navigate to System UI → UI Policies

Click New

Select the table (e.g., Incident)

Define the condition (Example: State = Resolved)

Save the UI Policy

Add UI Policy Action

Set the field as Mandatory

Example:

Make Resolution Notes mandatory when incident state becomes Resolved.

Creating Update Sets

Steps:

Navigate to System Update Sets → Local Update Sets

Click New

Enter name and description

Click Make Current

Perform configuration changes

Set update set state to Complete

Migrating Update Sets

Steps:

Export update set as XML

Import it into target instance

Preview update set

Resolve conflicts

Commit update set

6.PROJECT DEVELOPMENT PHASE

Project flow:

MILESTONE 1:update set

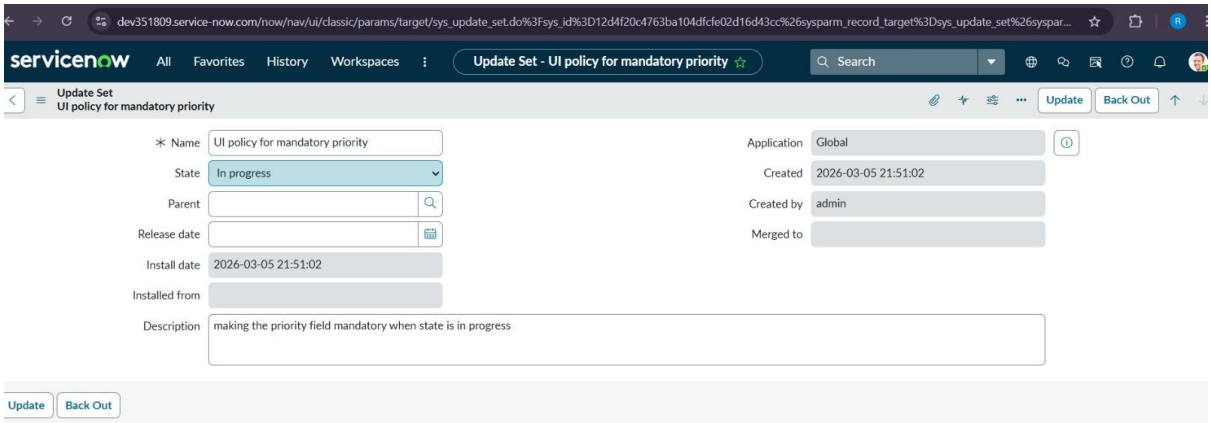
MILESTONE 2:UI policy

MILESTONE 3: Result

Milestone 1 : Update Sets

Activity 1: Create Update Set

1. Open service now.
2. Click on All >> search for Update Sets
3. Select Local Update Sets
4. Click on new
5. Fill the details below and make current.



6. Click on Submit

Milestone 2: UI policy

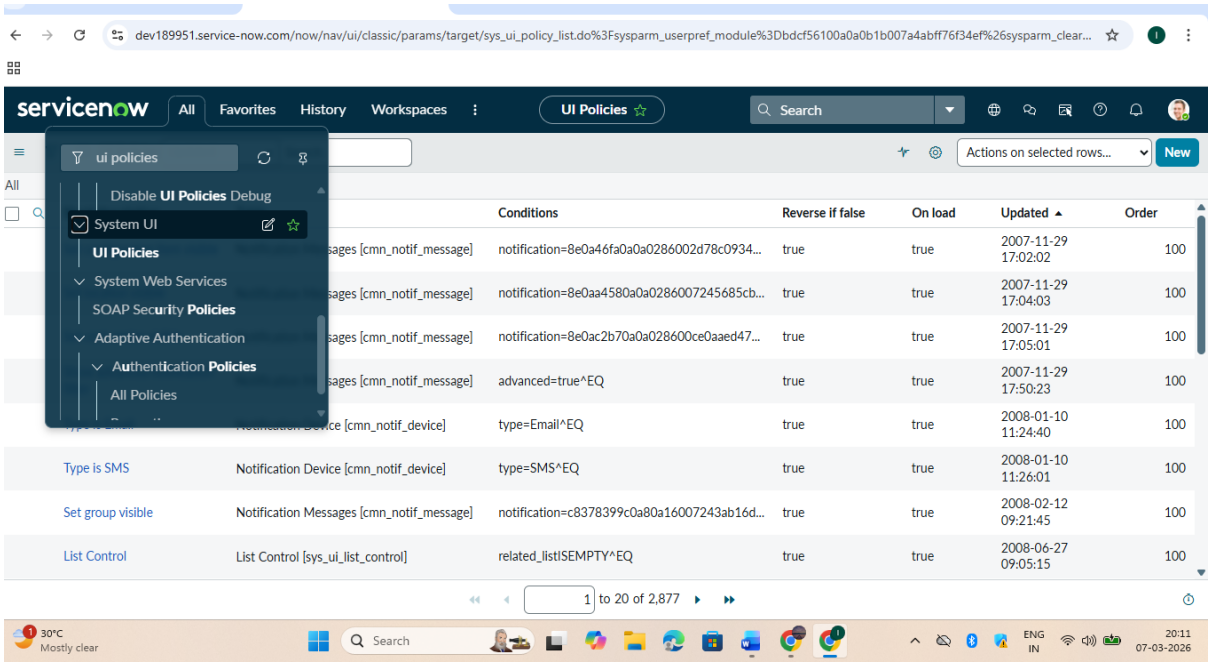
Activity 1: Create UI policy

1. Open service now.

2. Click on All >> search for ui policies

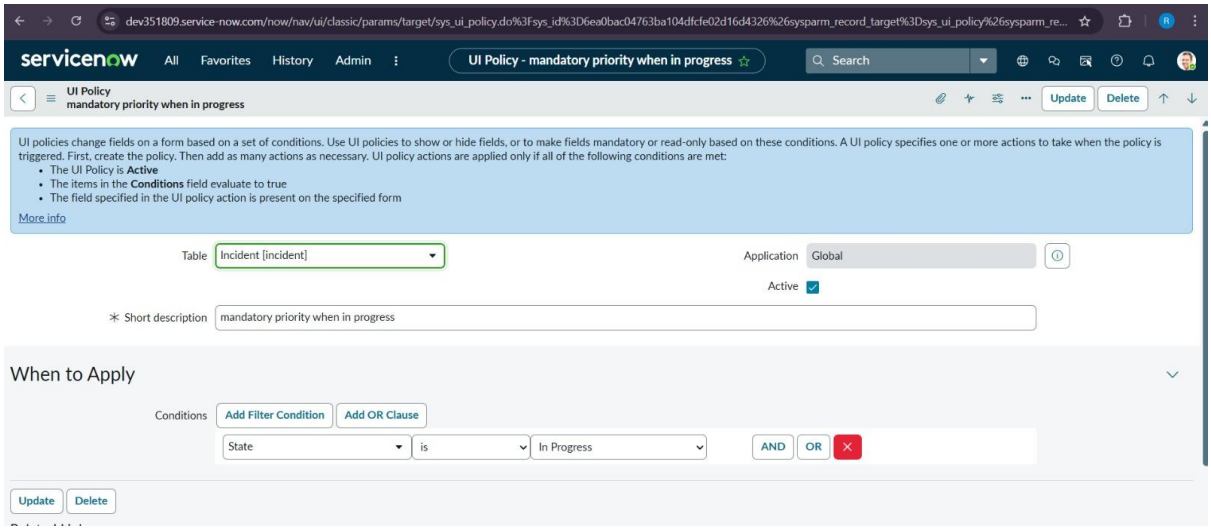
3. Navigate to System UI >> UI Policies

4. Click on new



5. Fill the following details and Save

6. Scroll Down and add ‘priority’ Field on Ui policy Actions



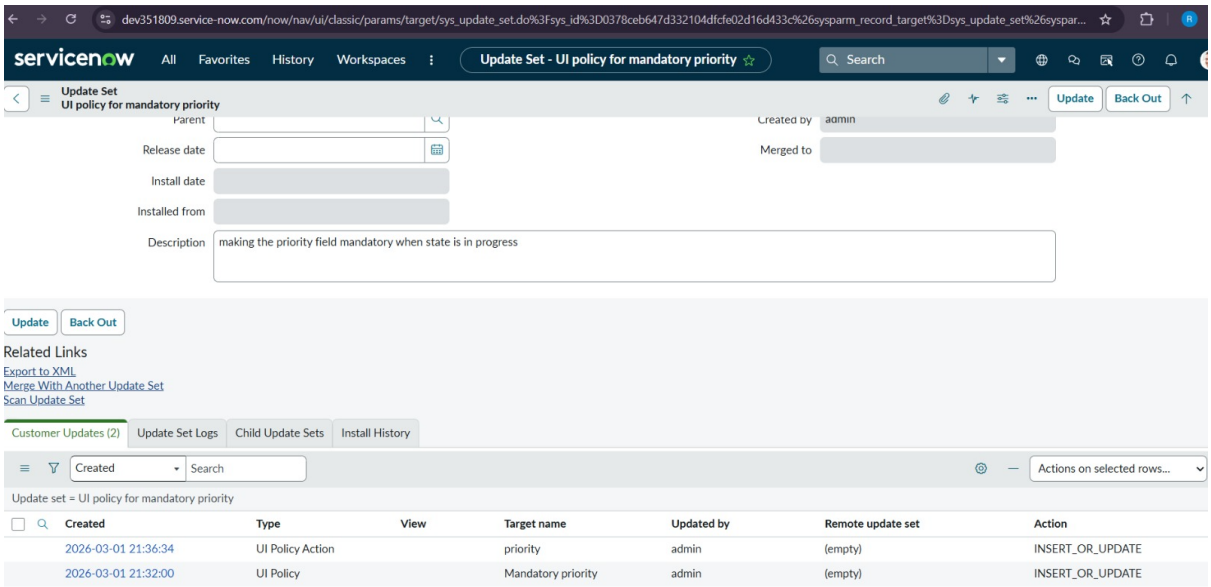
7. Click on submit.

Activity 2: Check Update Set

1. Navigate to the System Update Sets > Local Update Sets.

2. Open the ‘Ui policy for mandatory priority’ Update Set.

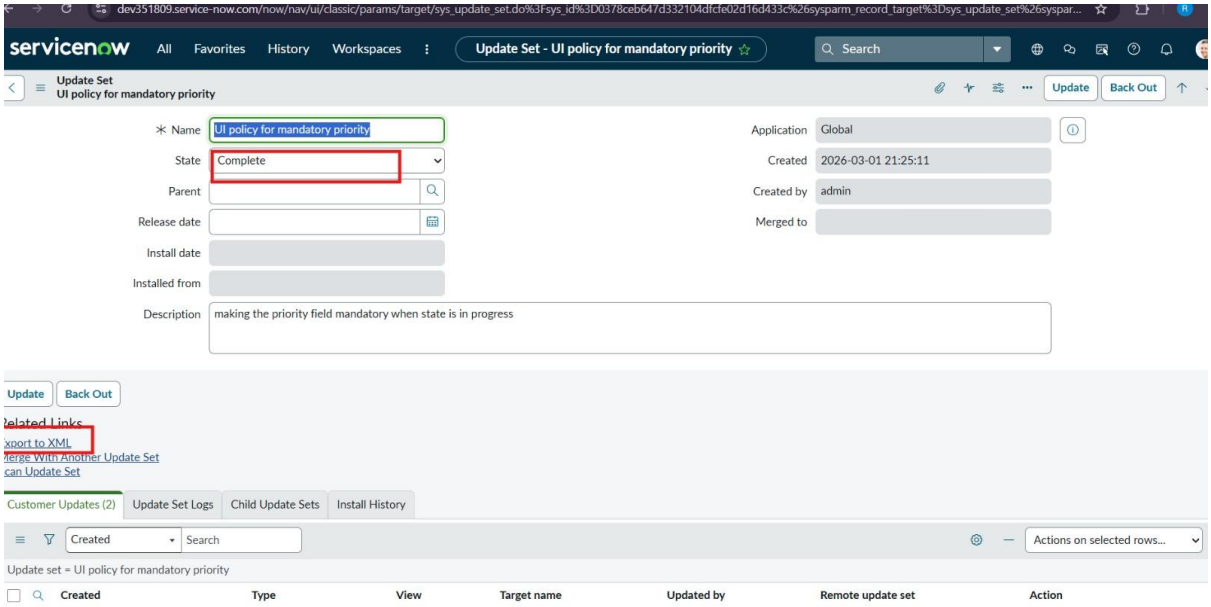
3. Verify that your UI Policy and UI Policy Action are listed.



Activity 3: Export and Import the Update Set

1. Set State to Complete and Click Export to XML on the Update Set record.

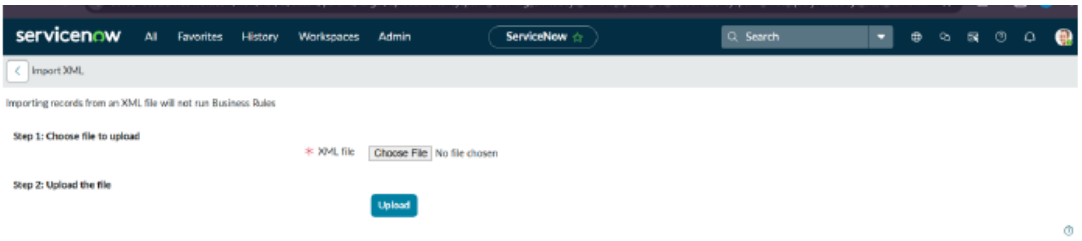
2. Download the XML file.



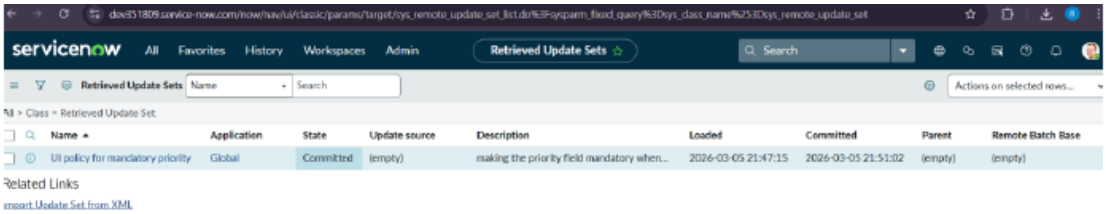
3. Now Open Another Instance to Export Update Set.

4. Go to System Update Sets >> Retrieved Update Sets.

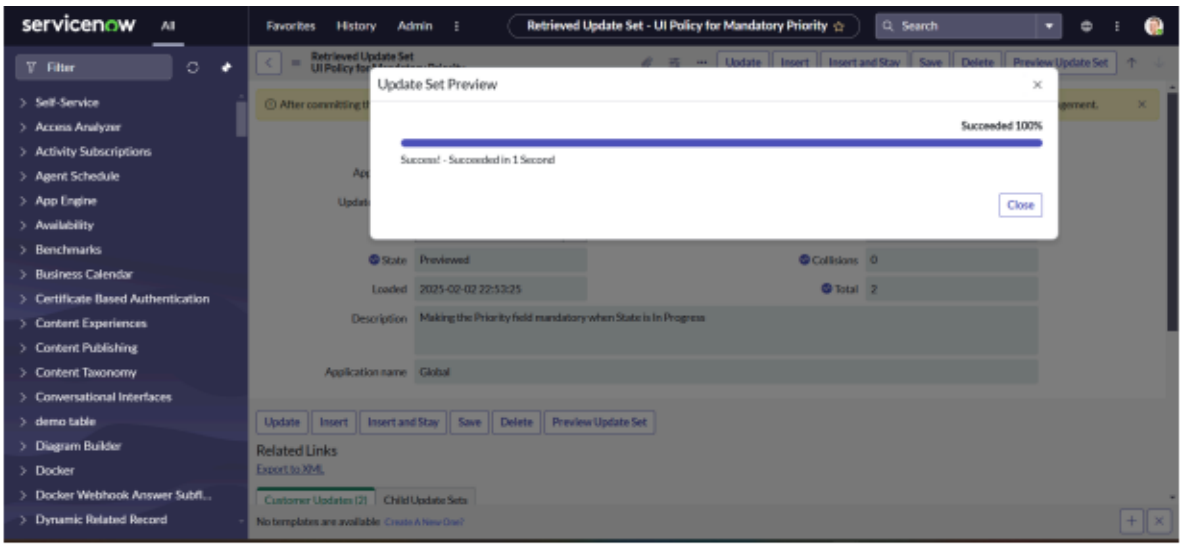
5. Click on the Import update set from XML and Select File and Upload



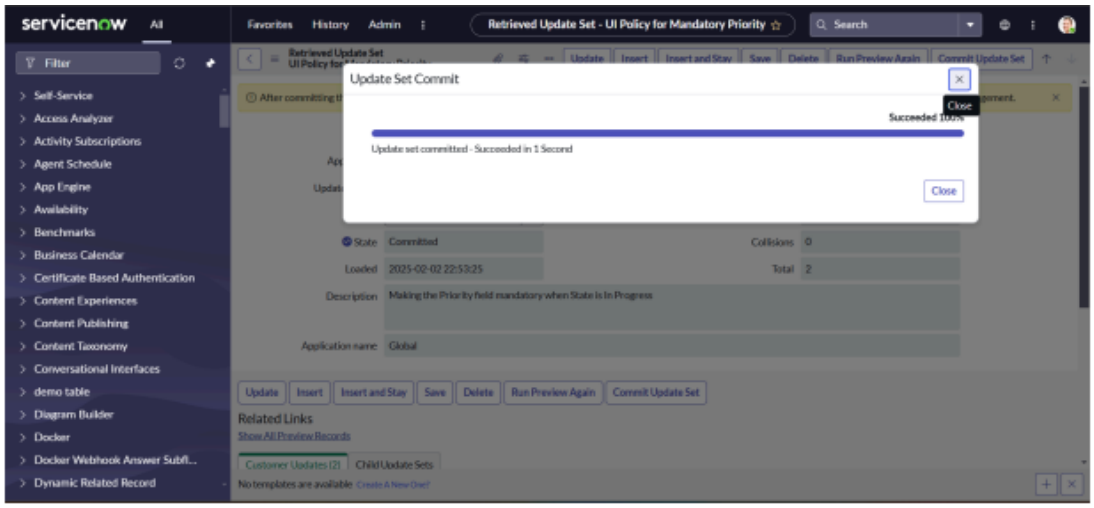
6. Open the update set that you have created in your instance here.



7. Preview the Update Set.



8. Commit the Update Set.



Milestone 3: Result

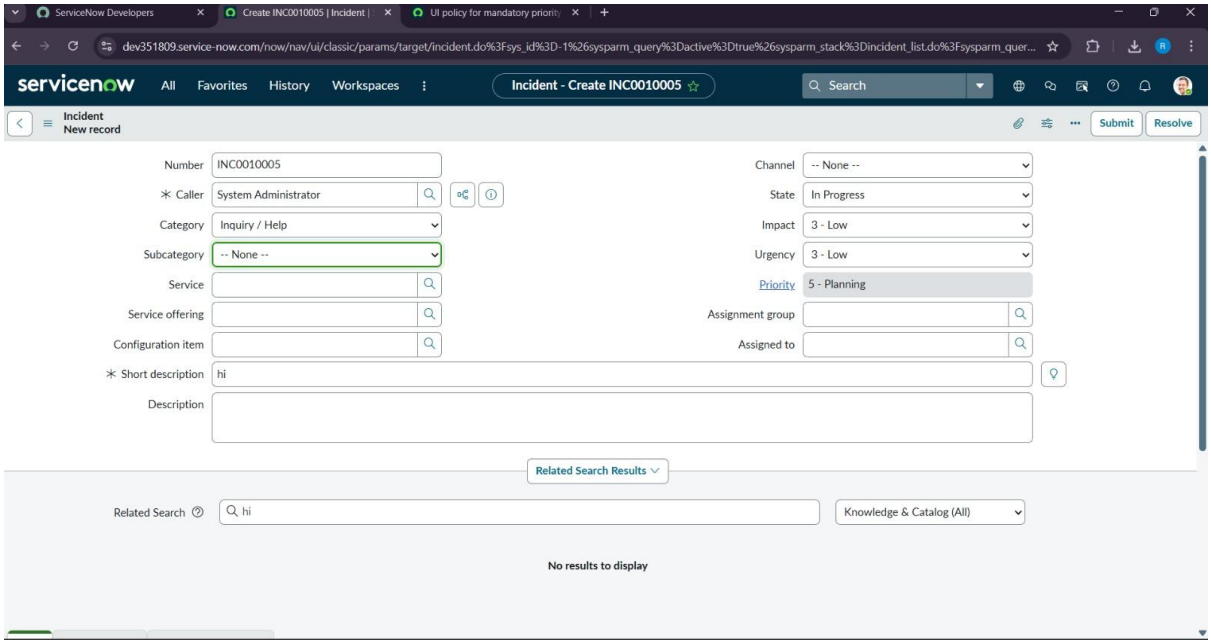
Activity 1: Testing Incident

1.Go to the incident module.

2.Open an existing incident or create a new one.

3.Change the State is In-progress

4.The Priority field should now be mandatory



7. FUNCTIONAL & PERFORMANCE TESTING

During the testing phase, the ServiceNow platform was evaluated to ensure that UI Policies and Update Sets function correctly.

Functional testing verified that UI Policies correctly enforce mandatory fields based on defined conditions. Forms were tested by creating records and checking whether the required fields were enforced properly.

Testing also confirmed that users could not submit records without filling mandatory fields when UI Policy conditions were triggered.

Update Set testing involved creating configuration changes in the development instance, exporting the update set as XML, and importing it into a testing instance. The preview feature was used to identify conflicts before committing the changes.

Performance testing ensured that UI Policies executed efficiently without slowing down form loading. Update Set migrations were also validated to ensure that configuration changes were successfully transferred across instances.

8. ADVANTAGES & DISADVANTAGES

Advantages

Improved data accuracy

Automated form validation

Better user experience

Safe migration of configuration changes

Efficient development management

Consistent configuration across environments

Disadvantages

Requires proper configuration

Migration conflicts may occur

Update Sets do not migrate data records

Large projects require careful management

Improper UI Policies may affect usability

9. CONCLUSION

The implementation of UI Policies and Update Sets in ServiceNow provides an efficient way to maintain data accuracy and manage configuration changes across multiple environments. UI Policies ensure that users provide all required information when submitting forms, reducing errors and improving workflow efficiency.

Update Sets enable developers to track and migrate configuration changes safely between development, testing, and production instances. This approach ensures consistent system behavior and simplifies deployment processes.

Overall, the project demonstrates how ServiceNow features can improve data integrity, system reliability, and development efficiency.

10. FUTURE SCOPE

Integration with automated workflows using Flow Designer

Advanced validation logic using client scripts

CI/CD integration for automated deployments

Enhanced reporting and monitoring tools

Integration with external enterprise systems

Improved security and role-based configuration management